Problem Statement
Solution : Enactor Bus Reservation System
- Before You Read
- Repo
- Backend Architecture
  - Architecure components diagram
  - Sequence Diagram (Across layers)
- API Reference
  - 1. Check Availability
  - 2. Create Reservation
- Development Environment
- How to Run
  - Deploy WAR file
  - Client App JAR file execution
  - Run the Web UI
  - Running the Docker
- Swagger Documentation
- Unit Tests
- Future Improvements

Problem Statement

Java Developer Assignment

Scenario

You need to develop a simple REST API to reserve tickets for a bus. The bus runs between four locations: **A, B,**
C, and D, making stops at each.

Requirements

1. **Bus Information:**
   - The bus has **40 seats**, arranged in **rows of 4 seats each** (e.g., 1A, 1B, …, 10D).
   - It makes a **one-way trip from A to D** and then returns **from D to A** once per day.
2. **Pricing:**

The ticket prices for different routes are:

- **A → B:** Rs. 50
- **A → C:** Rs. 100
- **A → D:** Rs. 150
- **B → C:** Rs. 50
- **B → D:** Rs. 100

• **C → D:** Rs. 50

Same prices are applicable for return journeys as well.

3. **Tasks:**

Need to implement **two REST APIs** using Java. (Preferred without using frameworks like Spring or
JAX-RS, but you can use a JSON library for handling JSON).

• **API 1: Check Availability & Price**
• Accepts: Number of passengers, origin, and destination
• Returns: Available seats and total price
• **API 2: Reserve Tickets**
• Accepts: Passenger count, origin, destination, and price confirmation
• Returns:
• Reservation or ticket number
• Assigned seat numbers
• Journey details (departure & arrival locations)
• Total price

4. **Additional Tasks:**

• **Provide unit tests** for the APIs.
• **Document** how to run the application and suggest improvements.

Restrictions:

• Using **core Java** is preferred over frameworks (ex. Spring, JAX-RS, etc.). But not mandatory.
• Store data **in memory** (no database required).
• Package the service in a **WAR file** that can be deployed.

Solution : Enactor Bus Reservation System

Before You Read

- This is a MVP version of app for the problem statement
- Using Core java + servlets as its specifically requested in the assignment
    - Output contains two modules
        - WAR file output(bus-reservation-system.war) that can be deployed in webserver
        - JAR file output (bus-client.jar) that can use to call services provided by the Api
- UI is in Basic HTML5 with JS (demo purpose only)
- Docker engine (v 4.63.0)

        - running on Window 11 PC
- Unit tests are added to cover all core logic

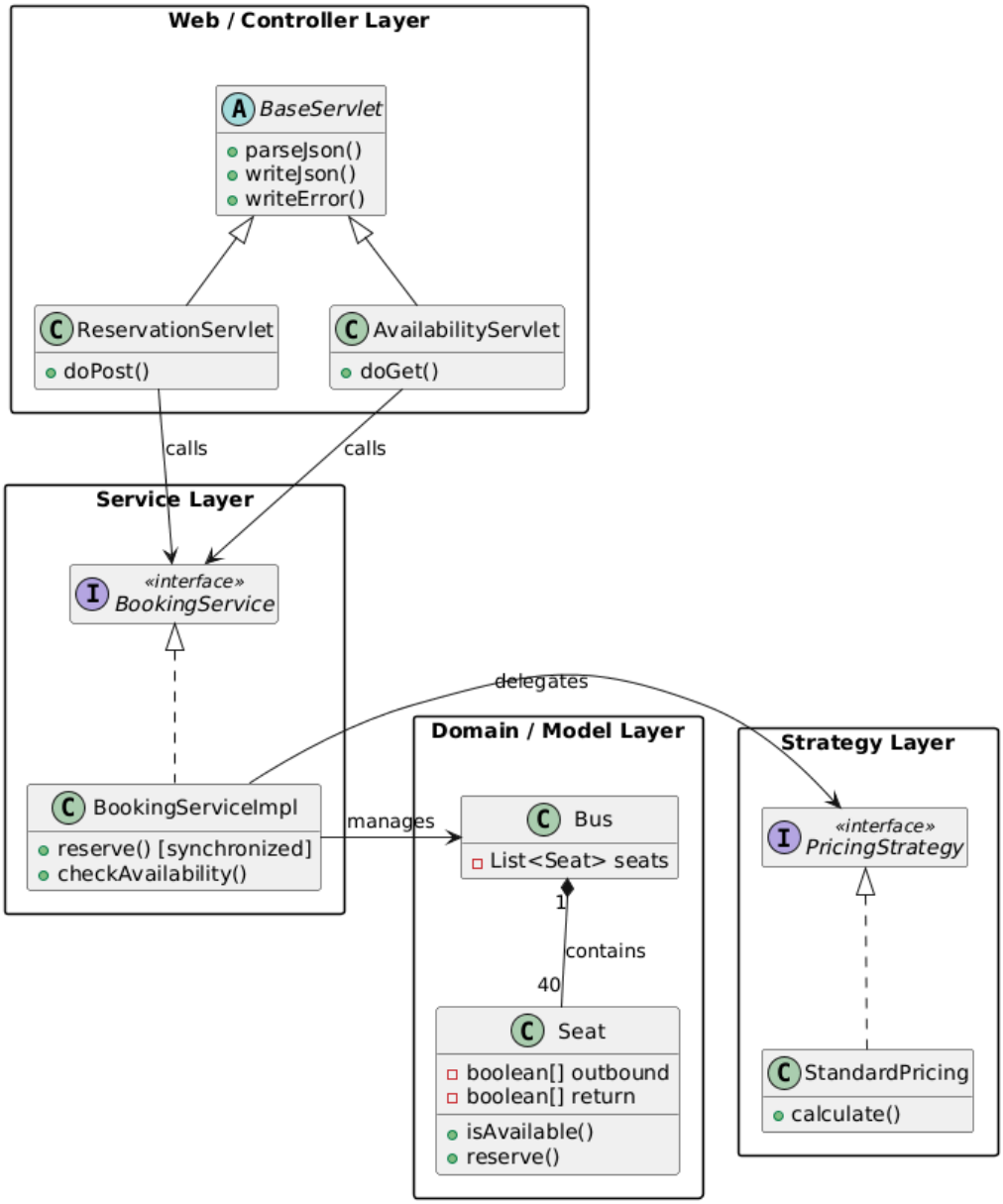
Repo

https://github.com/Erandauh/bus-reservation-system

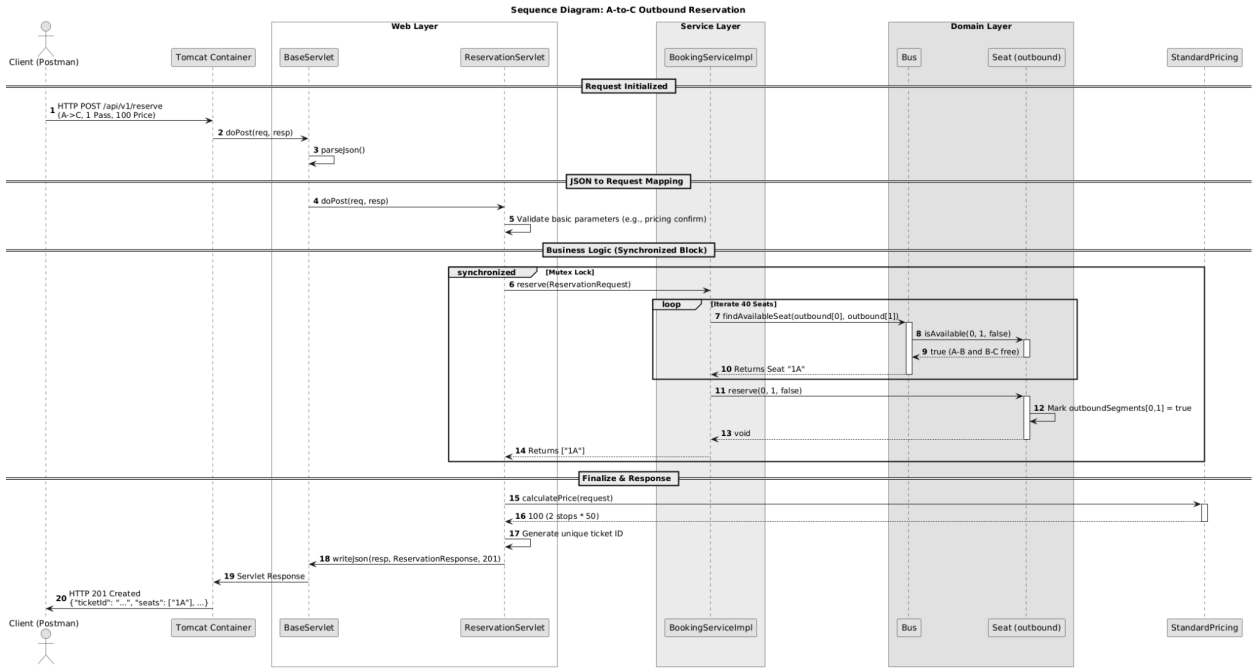
(It's public for now, please let me know once this evaluation is done, so I'll make it private)

Backend Architecture

Architecure components diagram



Sequence Diagram (Across layers)



API Reference

The Bus Reservation System exposes two primary REST endpoints. Both utilize standard HTTP status codes and JSON payloads.

1. Check Availability

Endpoint: `GET /api/v1/availability`

Description: Retrieves available seating and current pricing for a specific journey. This is a **read-only** operation.

- **Query Parameters:**
  - `origin` (String): Starting station (e.g., "A").
  - `destination` (String): End station (e.g., "C").
  - `passengers` (Integer, Optional): Default is 1.
- **Success Response:** `200 OK`
  - **Body:** Returns a list of available seat labels and the calculated total price.
- **Error Responses:**
  - `400 Bad Request`: Missing parameters or invalid numeric format for passengers.

---

2. Create Reservation

Endpoint: `POST /api/v1/reserve`

Description: Performs the booking action. It validates that the price hasn't changed since the user checked availability and atomically reserves the seats.

Request Body:
JSON
```
{
  "origin": "A",
  "destination": "C",
  "passengerCount": 2,
  "priceConfirmation": 200
}
```

- **Success Response:** `201 Created`
  - **Body:** Includes a unique `ticketId`, the assigned `seats`, and the `totalPrice`.
- **Error Responses:**
  - `400 Bad Request`: Data validation error or price mismatch.
  - `409 Conflict`: The requested seats are no longer available (Race condition handled).
  - `500 Internal Server Error`: Unexpected system failure.

Implementation Standards

- **Idempotency:** The `GET` endpoint is idempotent and has no side effects on the data.
- **Consistency:** The `POST` endpoint ensures atomicity; if 2 seats are requested and only 1 is left, the entire transaction fails to prevent partial bookings.
- **Content-Type:** All requests and responses are `application/json; charset=UTF-8`.

Development Environment

Windows 11 based PC
Intellij IDE
jdk-17.0.18.8-hotspot
Tomcat 11.0.18
Postman 11.85.1

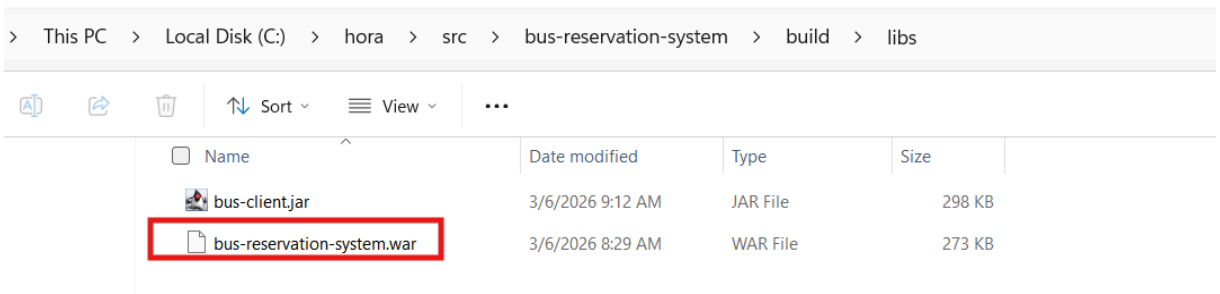
How to Run

Deploy WAR file

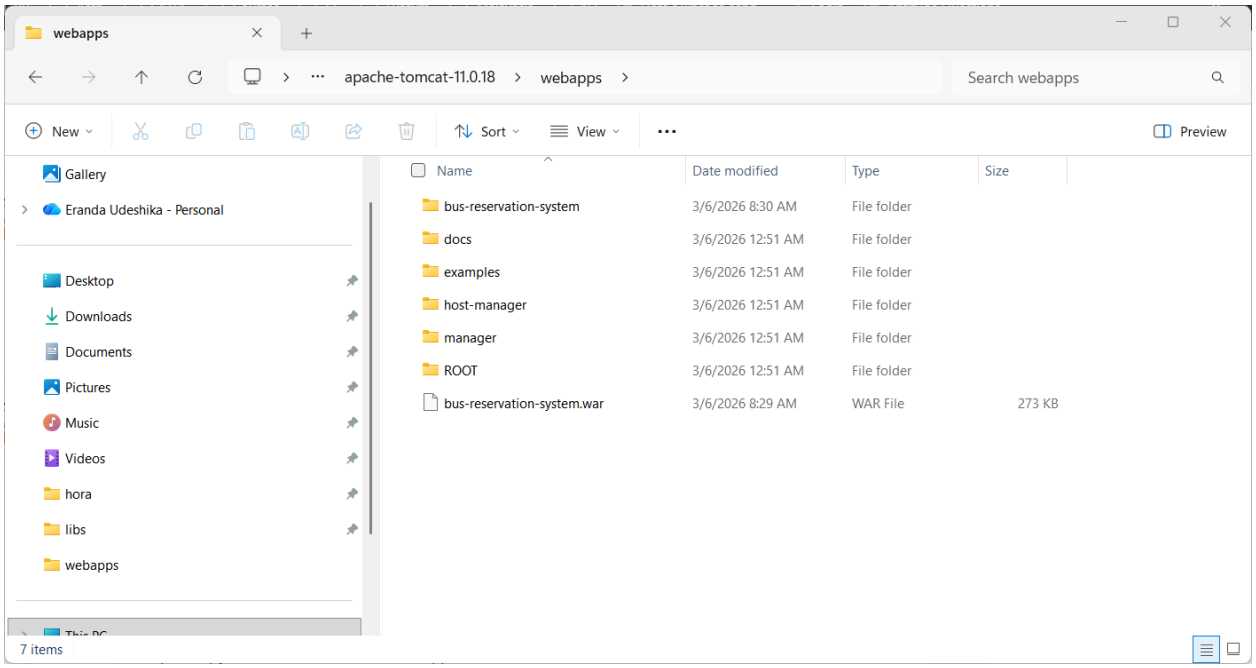
Step 1: Command to build war : ./gradlew clean war

```
PS C:\hora\src\bus-reservation-system> ./gradlew clean war
Starting a Gradle Daemon, 1 incompatible Daemon could not be reused, use --status for details

BUILD SUCCESSFUL in 18s
3 actionable tasks: 3 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.0.0/userguide/configuration_cache_enabling.html
```

Step 2: 'bus-reservation-system.war' file will be created in the '...\bus-reservation-system\build\libs'



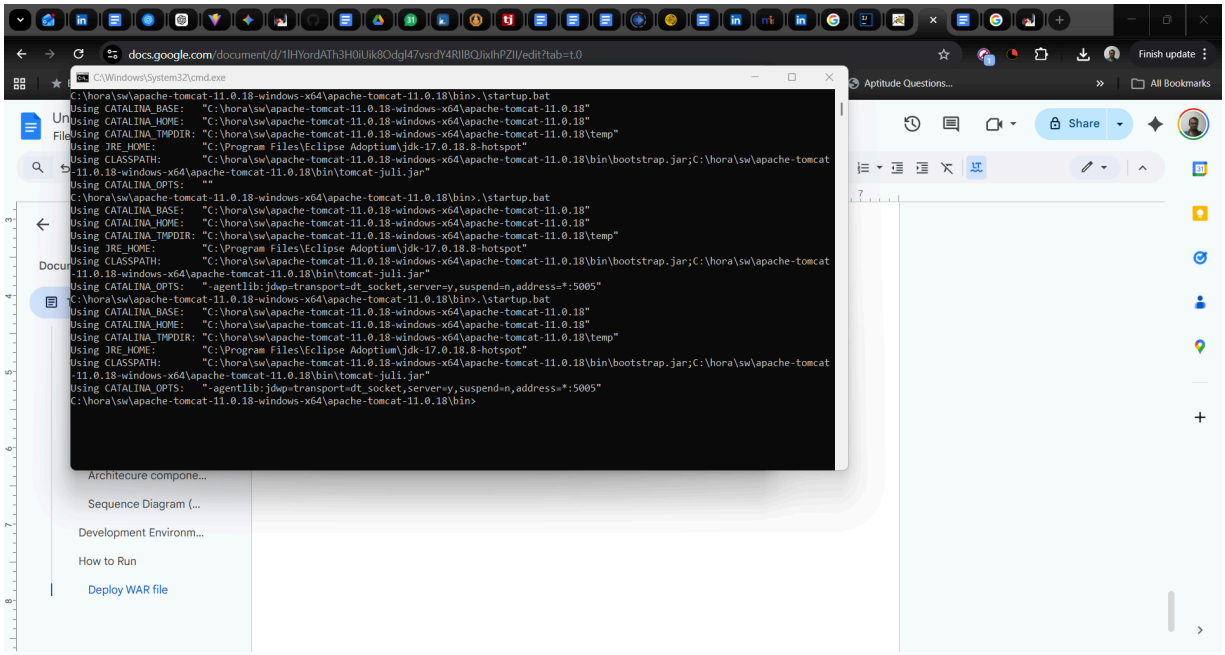
Step 3: copy the war file to Tomcat 'webapps' folder



Step 4: run the server

- *Go to bin folder of the tomcat and run .\startup.bat*

(this will deploy the web app in the server)



Step 4: go to postman and create reuquest

GET localhost:8080/bus-reservation-system/api/v1/availability?origin=A&destination=B&passengers=3

Or
import the following Curl

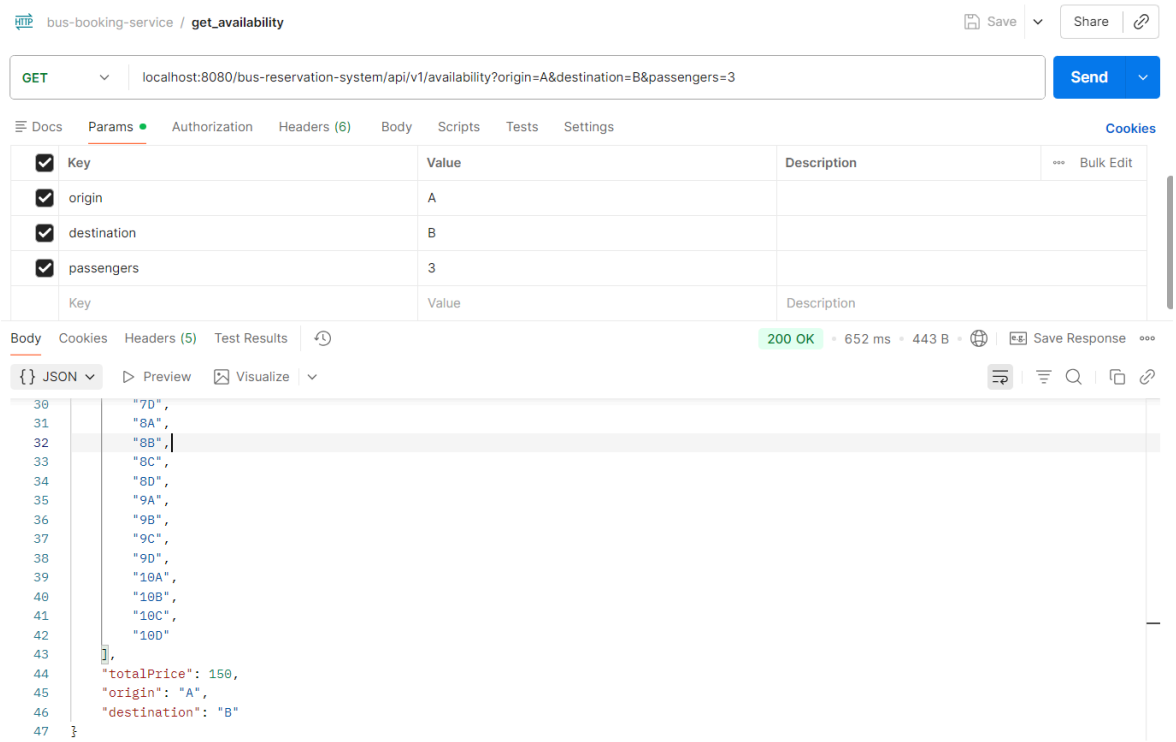
```
postman request 'localhost:8080/bus-reservation-system/api/v1/availability?origin=A&destination=B&passengers=3'
```

Additional info : get and import postman collection attached in the project repo itself
https://github.com/Erandauh/bus-reservation-system/blob/main/bus-booking-service.postman_collection.json

Step 4.1: Attach the debugger (intellij)

Add following lint to setenv.bat in the Tomcat bin folder (this will open the debug port for IDE to use with)

```
set "CATALINA_OPTS=-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005”
```



Step 5: test the other Api

```
postman request POST 'localhost:8080/bus-reservation-system/api/v1/reserve' \
  --header 'Content-Type: application/json' \
  --body '{
    "origin": "A",
    "destination": "C",
    "passengerCount": 2,
    "priceConfirmation": 200
}'
```

bus-booking-service / **make_reservation**

Save | Share

POST | localhost:8080/bus-reservation-system/api/v1/reserve | Send

Docs | Params | Authorization | Headers (8) | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
1 {
2     "origin": "A",
3     "destination": "C",
4     "passengerCount": 2,
5     "priceConfirmation": 200
6 }
```

Body | Cookies | Headers (5) | Test Results | 201 Created | 40 ms | 284 B | Save Response

JSON | Preview | Visualize

```
1  {
2      "reservationNumber": "08DBBE83",
3      "assignedSeats": [
4          "1A",
5          "1B"
6      ],
7      "origin": "A",
8      "destination": "C",
9      "totalPrice": 200
10 }
```

bus-booking-service / **make_reservation**

Save | Share

POST | localhost:8080/bus-reservation-system/api/v1/reserve | Send

Docs | Params | Auth | Headers (8) | Body | Scripts | Tests | Settings | Cookies

raw | JSON | Schema | Beautify

```
1 {
2     "origin": "A",
3     "destination": "C",
4     "passengerCount": 2,
5     "priceConfirmation": 2000
6 }
```

Body | 400 Bad Request | 184 ms | 211 B | Save Response

JSON | Preview | Debug with AI

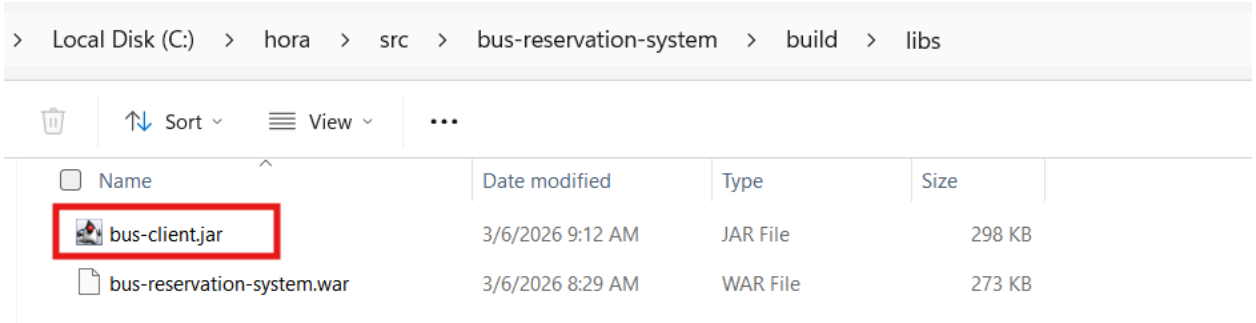
```
1 {
2     "error": "Invalid price entered 2000! expected price is 200"
3 }
```

Client App JAR file execution

Step 1: Build the jar with ./gradlew clientJar

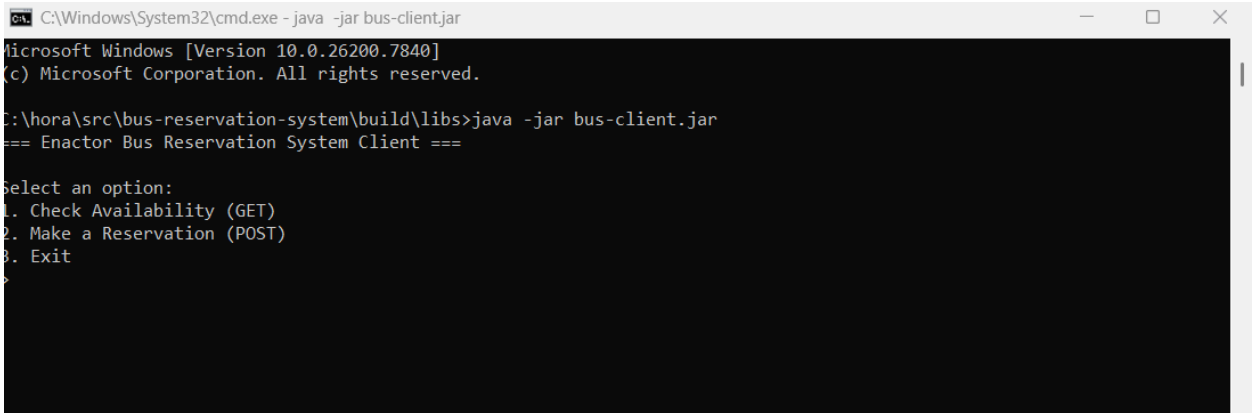
```
PS C:\hora\src\bus-reservation-system> ./gradlew  clientJar

BUILD SUCCESSFUL in 5s
2 actionable tasks: 1 executed, 1 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.0.0/userguide/configuration_cache_enabling.html
```

Step 2: Find jar in the 'bus-reservation-system\build\libs' folder



Step 3: Run the jar with

Java -jar bus-client.jar

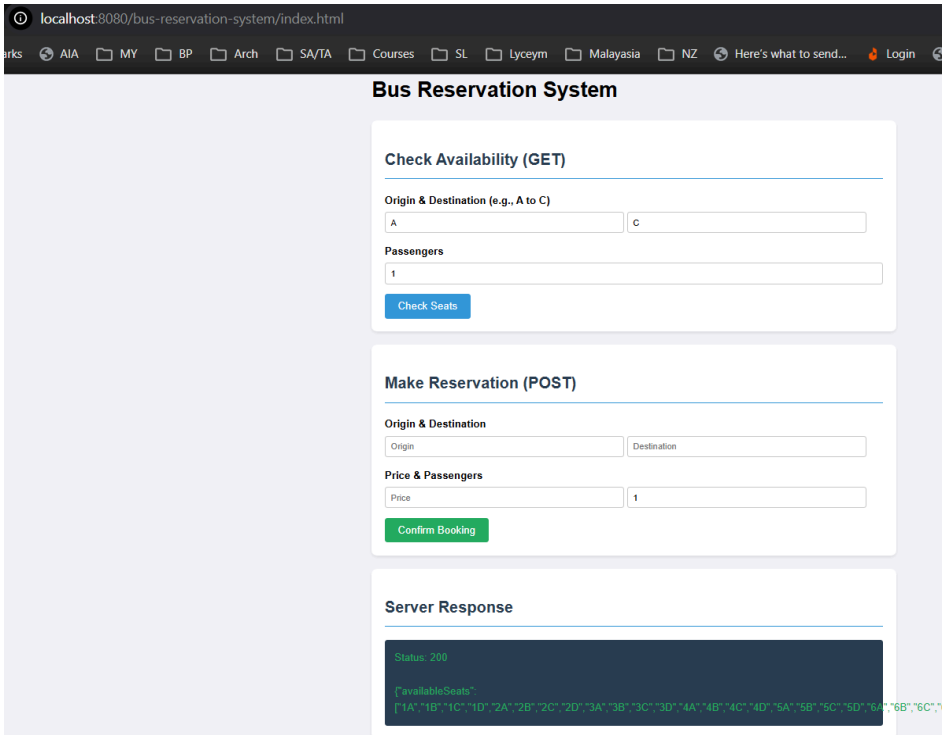


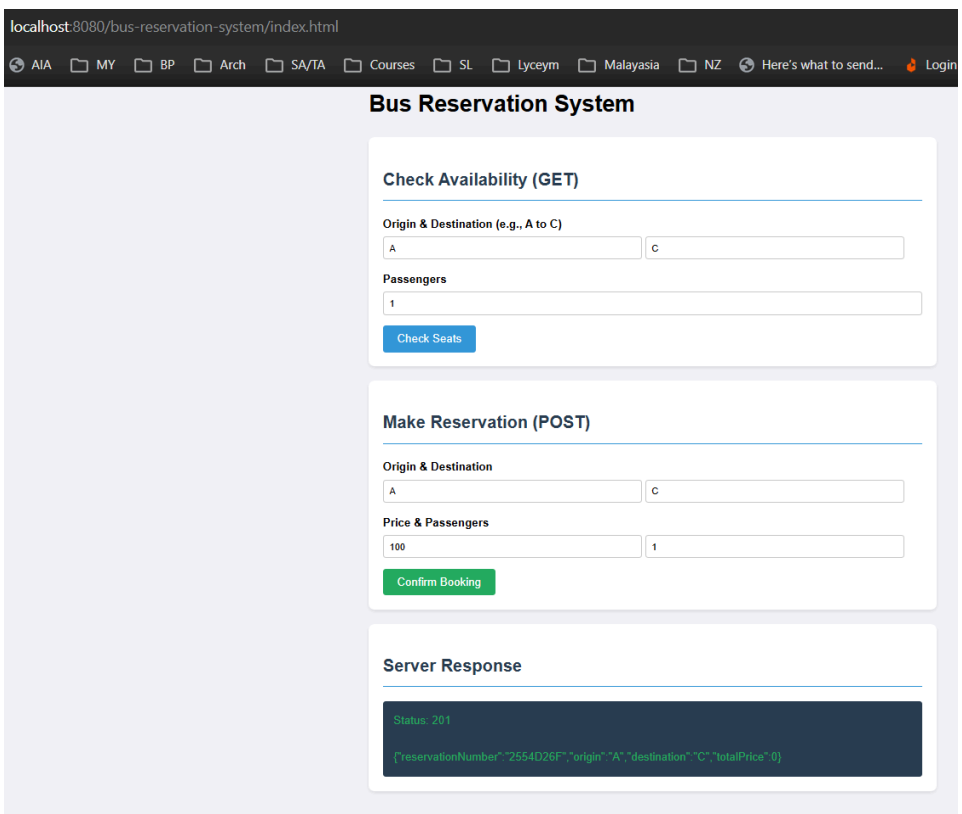
```
C:\Windows\System32\cmd.exe - java  -jar bus-client.jar
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\hora\src\bus-reservation-system\build\libs>java -jar bus-client.jar
=== Enactor Bus Reservation System Client ===

Select an option:
1. Check Availability (GET)
2. Make a Reservation (POST)
3. Exit
> 1
Enter Origin: A
Enter Destination: C
Enter Number of Passengers: 2

--- Server Response ---
Status: 200
Body: {"availableSeats":["3C","3D","4A","4B","4C","4D","5A","5B","5C","5D","6A","6B","6C","6
destination":"C"}
-----------------------

Select an option:
1. Check Availability (GET)
2. Make a Reservation (POST)
3. Exit
> 2
Enter Origin: A
Enter Destination: X
Enter Number of Passengers: 3
Enter Price to Pay: 1

--- Server Response ---
Status: 400
Body: {"error":"Invalid route from A to X"}
-----------------------

Select an option:
1. Check Availability (GET)
2. Make a Reservation (POST)
3. Exit
>
```

Run the Web UI

- NOTE: This UI is created Demo purpose only

Go to the browser and browse
http://localhost:8080/bus-reservation-system/index.html


localhost:8080/bus-reservation-system/index.html
AIA
MY
BP
Arch
SA/TA
Courses
SL
Lyceym
Malayasia
NZ
Here's what to send...
Login
Bus Reservation System
Check Availability (GET)
Origin & Destination (e.g., A to C)
A
C
Passengers
1
Check Seats
Make Reservation (POST)
Origin & Destination
Origin
Destination
Price & Passengers
Price
1
Confirm Booking
Server Response
Status: 200
{"availableSeats":
["1A","1B","1C","1D","2A","2B","2C","2D","3A","3B","3C","3D","4A","4B","4C","4D","5A","5B","5C","5D","6A","6B","6C",


localhost:8080/bus-reservation-system/index.html
AIA
MY
BP
Arch
SA/TA
Courses
SL
Lyceym
Malayasia
NZ
Here's what to send...
Login
Bus Reservation System
Check Availability (GET)
Origin & Destination (e.g., A to C)
A
C
Passengers
1
Check Seats
Make Reservation (POST)
Origin & Destination
A
C
Price & Passengers
100
1
Confirm Booking
Server Response
Status: 201
{"reservationNumber":"2554D26F","origin":"A","destination":"C","totalPrice":0}

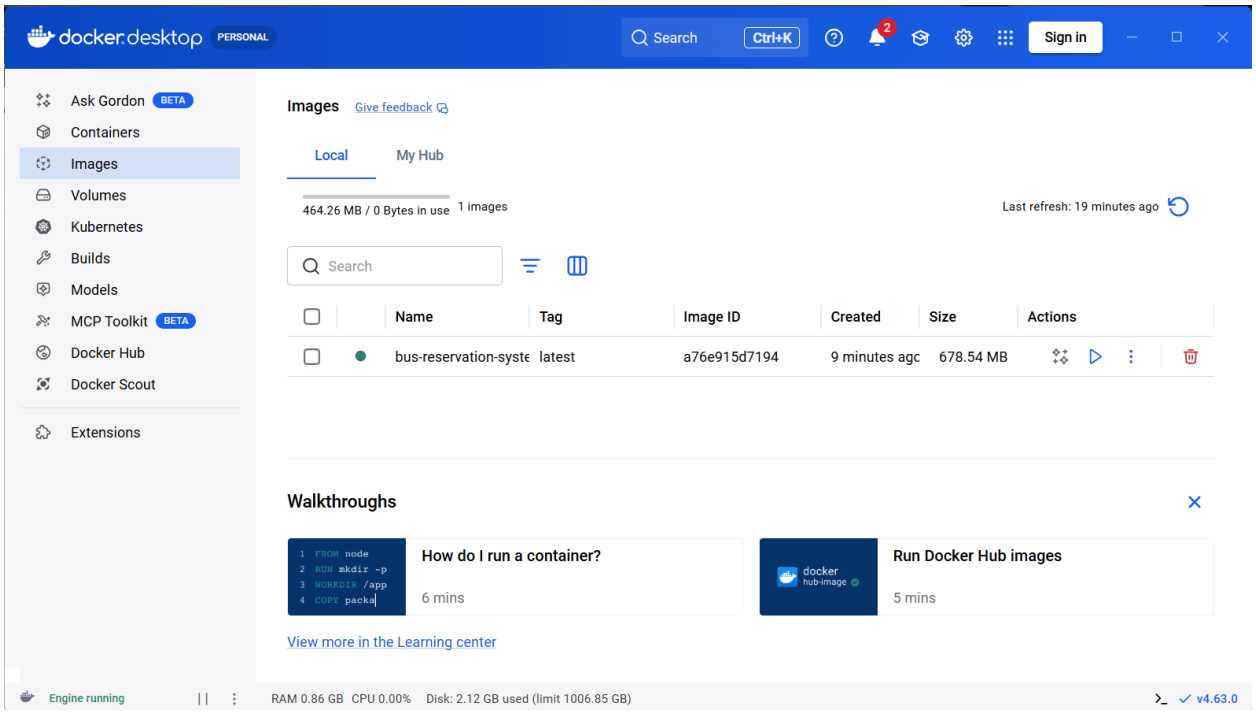
Running the Docker

Run command in the terminal : *docker build -t bus-reservation-system*

This will create the docker image

```
PS C:\hora\src\bus-reservation-system> docker build -t bus-reservation-system .
[+] Building 214.4s (4/7)                                                    docker:desktop-linux
[+] Building 214.6s (4/7)                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                       0.1s
[+] Building 214.7s (4/7)                                                    docker:desktop-linux
```

Then run the docker image in docker UI



Api is available in the

http://localhost:8080/api/v1/availability?origin=A&destination=C

Note: we launch from the root dir of tomcat now

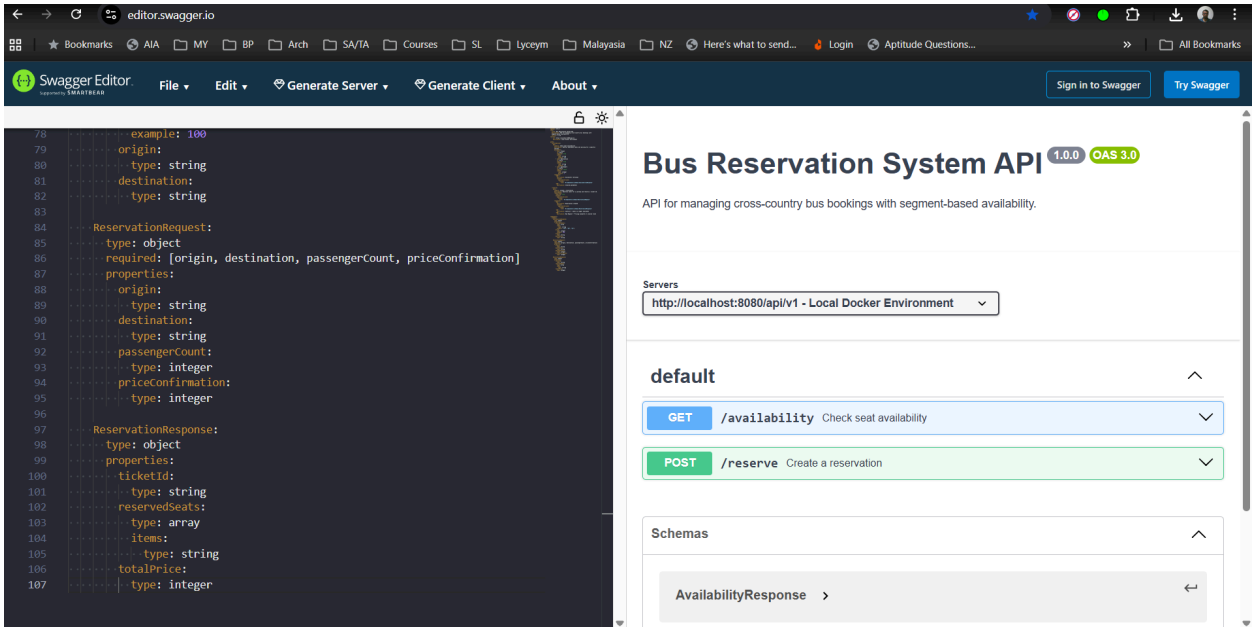
localhost:8080/api/v1/availability?origin=A&destination=C

Bookmarks AIA MY BP Arch SA/TA Courses


```
{
  "availableSeats": [
    "1A",
    "1B",
    "1C",
    "1D",
    "2A",
    "2B",
    "2C",
    "2D",
    "3A",
    "3B",
    "3C",
    "3D",
    "4A",
    "4B",
    "4C",
    "4D",
    "5A",
    "5B",
    "5C",
    "5D",
    "6A",
    "6B",
    "6C",
    "6D",
    "7A",
    "7B",
    "7C",
    "7D",
    "8A",
    "8B",
    "8C",
    "8D",
    "9A",
    "9B",
    "9C",
    "9D",
    "10A",
    "10B",
    "10C",
    "10D"
  ],
  "totalPrice": 100,
  "origin": "A",
  "destination": "C"
}
```

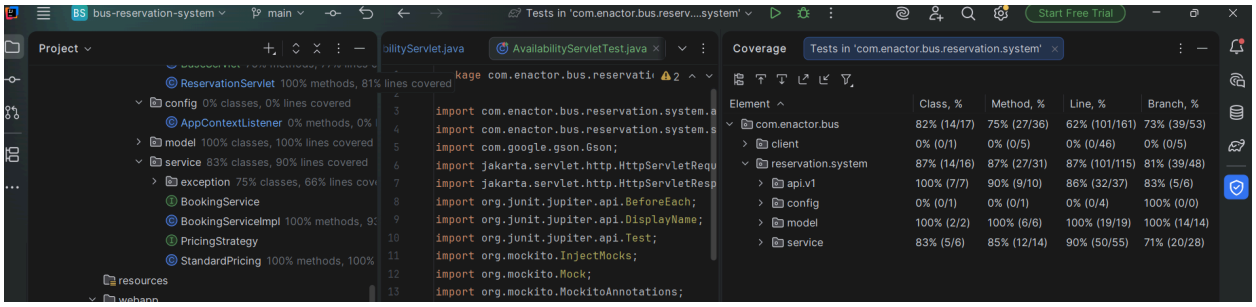

Swagger Documentation

API Documentation (Included manual swagger doc as I do not use any frameworks as Spring)
The API is documented using the **OpenAPI 3.0 (Swagger)** standard.
- Can view the specification in `docs/bus-reservation-api.yaml`.
- To view it interactively, paste the contents into the [Swagger Editor](https://editor.swagger.io/).



Unit Tests

All core logic is covered with tests



Future Improvements

1. **Introduce a Persistence Layer (Database Integration)**

Currently, all data (Bus/Seat) is stored in RAM. If the Server/Docker container restarts, all bookings are lost.

Proposed Solution: Migrate the in-memory collections to a relational database like MySQL and implement the Repository Pattern and use JPA/Hibernate for Object-Relational Mapping (ORM).

2. **Dstributed Locking (Horizontal Scaling)**

Current thread-safety impl relies on the synchronized (in memory). This works for one server, but once we need to run in multipods env, they won't know about each other's locks.

- **Proposed Solution:** Use Redis lock that allows the app to scale horizontally across multiple servers while still preventing double-booking.

3. **Secure the Api**

The API is currently public. Anyone can book or check availability. Use Authentication and Authorization

- **Proposed Solution:** Implement JWT (JSON Web Tokens) or OAuth2

4. **Logging**

Currently no logging functionality. This makes almost impossible to troubleshoot in the production environments

- **Proposed Solution:** Integrate SLF4J/Logback with an ELK stack

5. **Build and interactive HLL (Hyper Local Layer) in Web, Android and IOS**

- **Proposed Solution:** Current WebUI is only with basic html5 and JS. Build a **React** or **Angular** single-page application (SPA) tha can provide a visual seat-map (e.g., a grid of 40 seats) where users can click to select their spot(user friendly).